

2.0 - CONVOLUTIONAL NEURAL NETWORKS

CHAPTER INDEX

- ☐ [CONVOLUTIONAL NEURAL NETWORKS](#)
 - [CONVOLUTIONS](#)
 - [CNN ARCHITECTURE](#)
 - [INPUT CHANNELS](#)
 - [VOLUMES AND FEATURE MAPS](#)
 - [POOLING LAYERS](#)
 - [CNNs AS TRANSLATION INVARIANT, CONSTRAINED MLPs](#)
- ☐ [SOME CNN EXAMPLES](#)
 - [GOOGLNET](#)
 - [RESNET](#)
 - [DENSENET](#)
 - [ENZYNET](#)
- ☐ [OTHER CONCEPTS](#)
 - [TRANSFER LEARNING](#)
 - [AUTO ENCODER](#)
 - [ADVERSIAL EXAMPLES](#)

CONVOLUTIONAL NEURAL NETWORKS

CNNs are a kind of **deep neural networks** that began achieving **good performance** in 1998 thanks to **Yann LeCun**'s work in the context of **small datasets** (e.g. character recognition).

The **true change of pace** happened in **2012** when **AlexNet** won the **ImageNet competition** with a *significant margin* compared to *other competitors*.

In practice, AlexNet didn't introduce any significant innovation to LeCun's CNNs; its success was mostly related to two factors:

1. Improvements in **computational power** of computers, especially GPUs, which could finally satisfy **computational speed**, **memory availability** and **memory bandwidth** required to train these models.
2. **Availability of large labeled datasets**: the *superiority of deep learning techniques* over other approaches is *clearer when large amounts of training data are available*.

At this point, we've given enough context to finally start diving into **how CNNs work**, and thus also **why they revolutionised deep learning**.

CONVOLUTIONS

The **key idea** behind **CNNs** is to **implement convolutions** in the neural network. Convolutions are among the most important **image processing operations** through which **digital filters are applied**.

Intuitively, a *digital filter* (a *mask of weights*) is "**scrolled**" over different **input positions** and **output values** are generated by computing the **scalar product between the mask and each portion of the input** (each portion is of the same size as the input).

Mathematically, in the case of image processing, the **inputs** and **filters** are **2D arrays**. Thus, **convolutions used in CNNs** are defined as follows:

🔖 CONVOLUTION (CNNs)

Let I be the **input image**, of size $w_i \times h_i$; and K be the **filter** (or *kernel*), of size $w_k \times h_k$. Then the **convolution of I with K** is defined as follows:

$$\text{Conv}(I, K)(i, j) = I * K(i, j) := \sum_{m=1}^{w_k} \sum_{n=1}^{h_k} I(i + m, j + n) \cdot K(m, n)$$

Where the indices i, j refer to the **output dimensions** and we have:

$$i = 1, \dots, w_i - w_k + 1, j = 1, \dots, h_i - h_k + 1$$

✎ NOTE

In reality, convolutions are defined with a **flipped version of the kernel** used above: mathematically, this is equivalent to using $I(i - m, j - n)$ instead.

What we defined above is actually the **cross-correlation** of I and K , but it's **called convolution** for convenience since *flipping the weights doesn't add representational power anyway*.

The following image helps gain a better understanding of what a convolution is doing:

Input image

9	4	1	2	2
1	1	1	0	4
1	2	1	0	4
1	0	0	2	1
9	6	7	4	1

Filter

0	2	1
4	1	0
1	0	1

Output array

16		

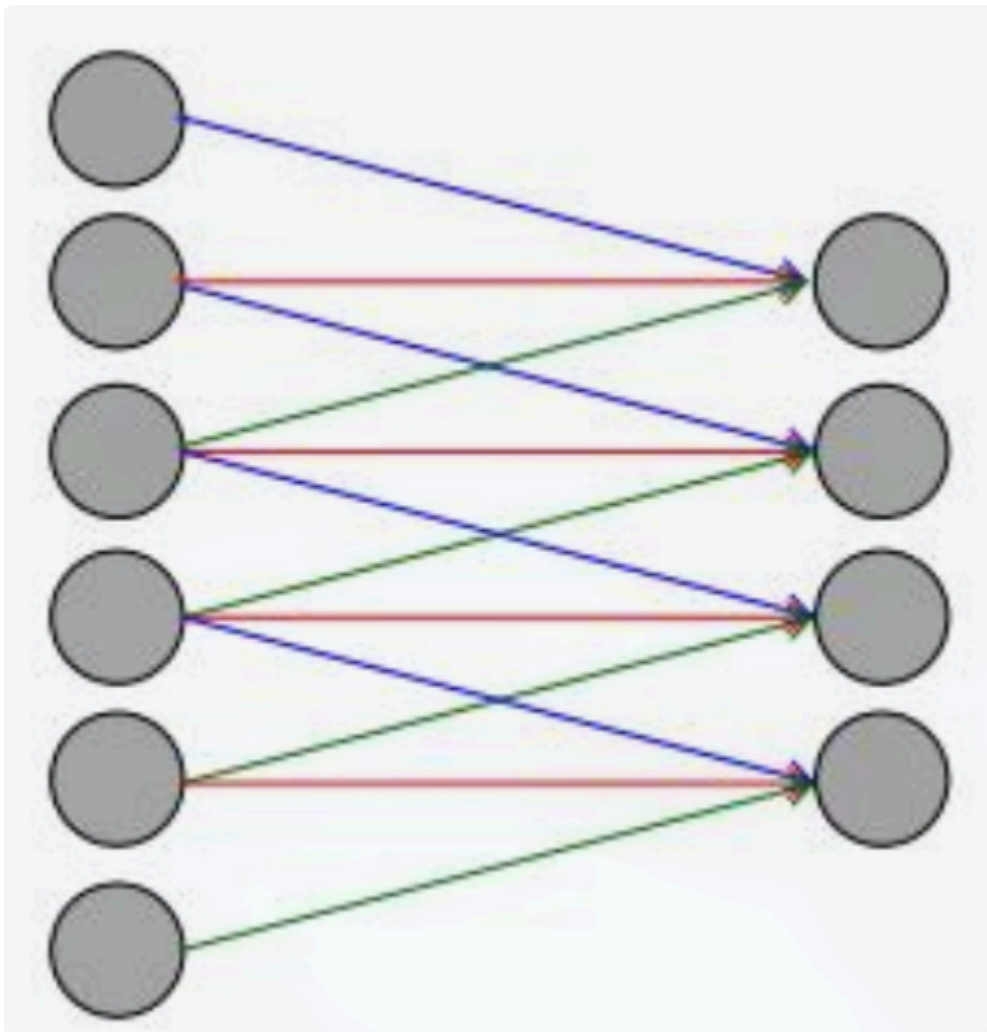
$$\begin{aligned}\text{Output}[0][0] &= (9*0) + (4*2) + (1*4) \\ &+ (1*1) + (1*0) + (1*1) + (2*0) + (1*1) \\ &= 0 + 8 + 1 + 4 + 1 + 0 + 1 + 0 + 1 \\ &= 16\end{aligned}$$

We can see that the **kernel "filters"** each portion of the input to produce a "condensed" output.

CNN ARCHITECTURE

Now that we know how convolutions work, we can show **how they are implemented in a neural network**.

While neurons in an MLP are *fully connected*, in a **CNN** they are only "**locally connected**" and **some weights are shared** (see figure below).



Notice that this is a *direct implementation of the convolutions*. To better understand why, consider the following observation:

💡 OBSERVATION

Notice that **each neuron on the right is connected to only 3 neurons of the previous layer**.

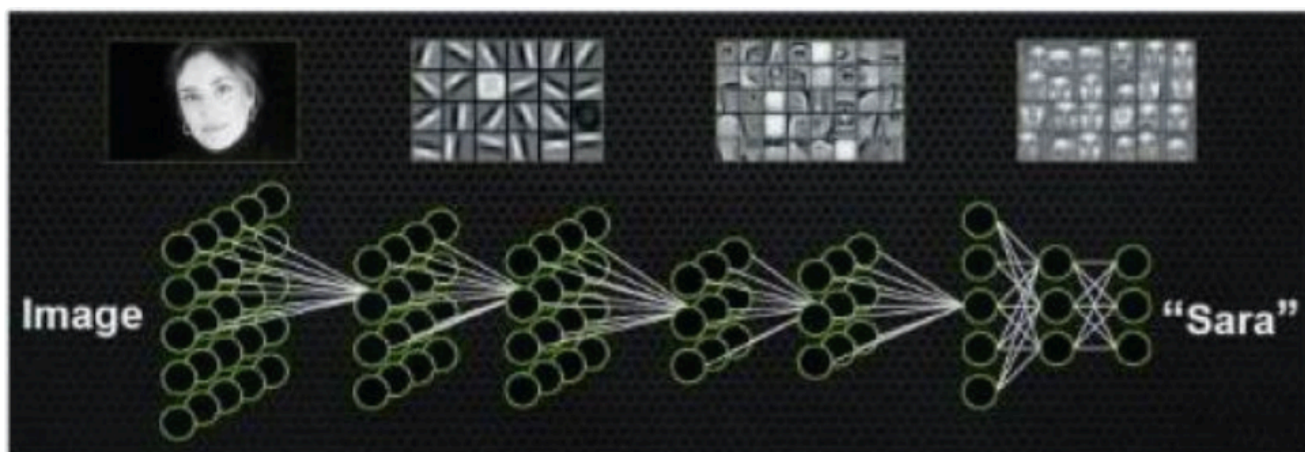
Furthermore, **weights of the same colour share the same value**: we thus have a **set of 3 weights** that is "**scrolled**" over the inputs.

This set of weights is precisely the **kernel** used for the convolution.

The above observation also helps us understand the **key motivation** behind the introduction of CNNs: with **only local connections** and **shared weights**, the **number of parameters is greatly reduced** (in the example, *12 total connections* and *3 distinct weights* as opposed to *24 connections* and *24 distinct weights* in a MLP with the same number of neurons), thus making **computation less expensive and more efficient**.

As for the model's architecture, CNNs usually comprise a **hierarchy of convolutional layers** (with the characteristics explained above), followed by **fully connected layers** that operate as an *MLP classifier*.

The *convolutional layers* perform **feature extraction** and the *final layers* **classify the extracted features**:



Notice that the **convolutional layers** are represented as **2D arrays of neurons** (and not just 1D columns). To be even more precise, each layer is actually a **3D volume**, as we have to consider the dimension given by the **input channels**: see > [INPUT CHANNELS](#) and > [VOLUMES, FILTERS AND FEATURE MAPS](#) for more details.

INPUT CHANNELS

As readers may already know, **each pixel** in an image can be **represented by multiple numbers**, not just one: these are called **channels**.

For example, a **greyscale** image only has **one channel**, while an **RGB image** has **3 channels** (*red, green, blue*).

In general, the number of channels can also be higher.

NOTE

Because CNNs were initially intended to work with images, the **"standard" input format** is a **tensor** (think of it as a *multi-dimensional array*) with dimensions:

$$\text{height} \times \text{width} \times \text{channels}$$

When **batches of data** are processed simultaneously, this becomes:

$$\text{batch size} \times \text{height} \times \text{width} \times \text{channels}$$

In any case, **non image data** can be **converted to a tensor** so it can be fed to the CNN.

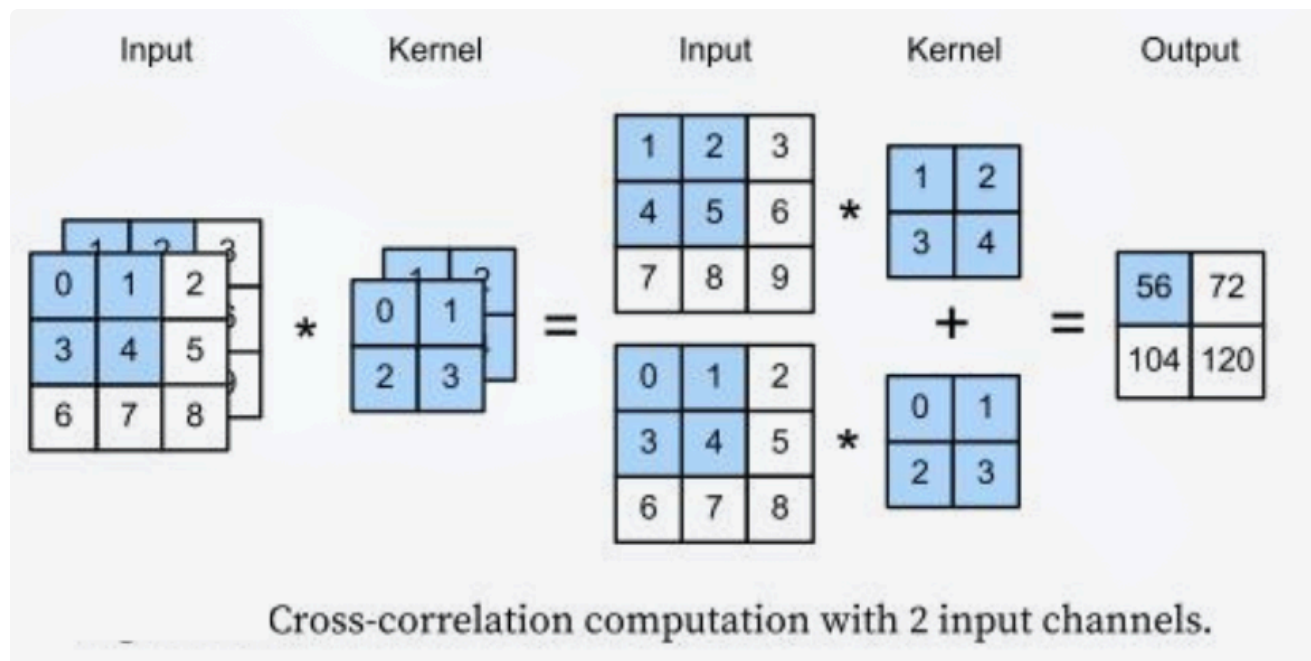
When input data contains *multiple channels*, we need to construct a convolutional **kernel** with the **same number of input channels** to perform the cross-correlation operations.

That is, if the input data has c_i channels and our **kernel's window shape** is $k_h \times k_w$, we need to **concatenate** c_i of these matrices to create a **tensor** of dimensions:

$$c_i \times k_h \times k_w$$

Since both the *input* and the *convolution kernel* have c_i channels, we can perform a **cross-correlation operation** on the *two-dimensional tensor* of the input and the *two-dimensional tensor* of the kernel **for each channel**, adding c_i **results together** to yield a simple *two-dimensional tensor*.

Consider the example in the figure to understand it better:



In the above example, we only have **one output channel**; but what happens when we have **multiple output channels**?

MULTIPLE OUTPUT CHANNELS

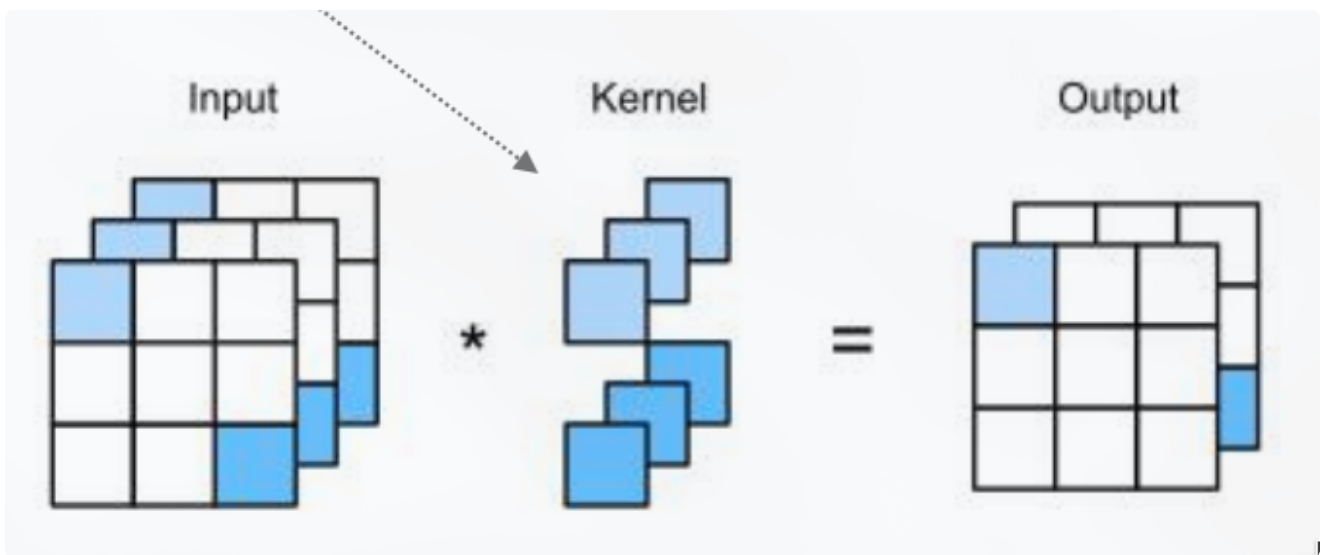
If we have c_o output channels, we **need just as many kernels**, each one **computing the output for a specific channel**.

Now, it is worth taking a look at **1 x 1 convolutions**.

At first, they don't seem to make sense: after all, a **convolution correlates adjacent pixels**, but a **1 x 1 kernel obviously doesn't**, and yet, they are often used in the designs of complex deep networks.

Because the minimum window is used, we're **losing the ability** of larger convolutional layers to **recognise patterns** consisting of **interactions among adjacent elements**; the **only computation** of the 1×1 convolution occurs in the **channel dimension**.

Consider the following figure for clarity:



In the above example, **two 1x1 convolution kernels** are used to **map 3 input channels into 2 output channels**.

🔗 INTERPRETATION

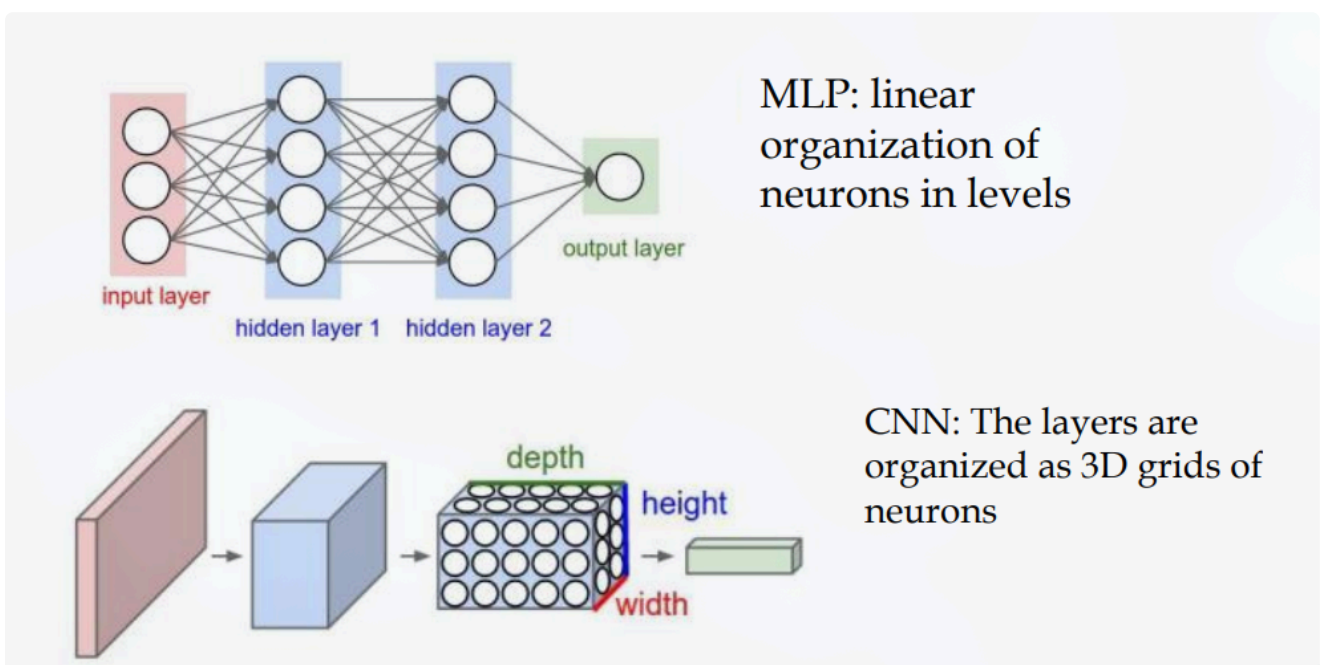
We can think of the 1×1 convolutional layer as a **fully connected layer applied at every pixel location** to *transform* c_i input values into c_o output values.

Because this is still a *convolutional layer*, the **weights are tied across pixel location**.

Thus, a 1×1 convolutional layer requires $c_o \times c_i$ **weights** (plus the bias).

VOLUMES AND FEATURE MAPS

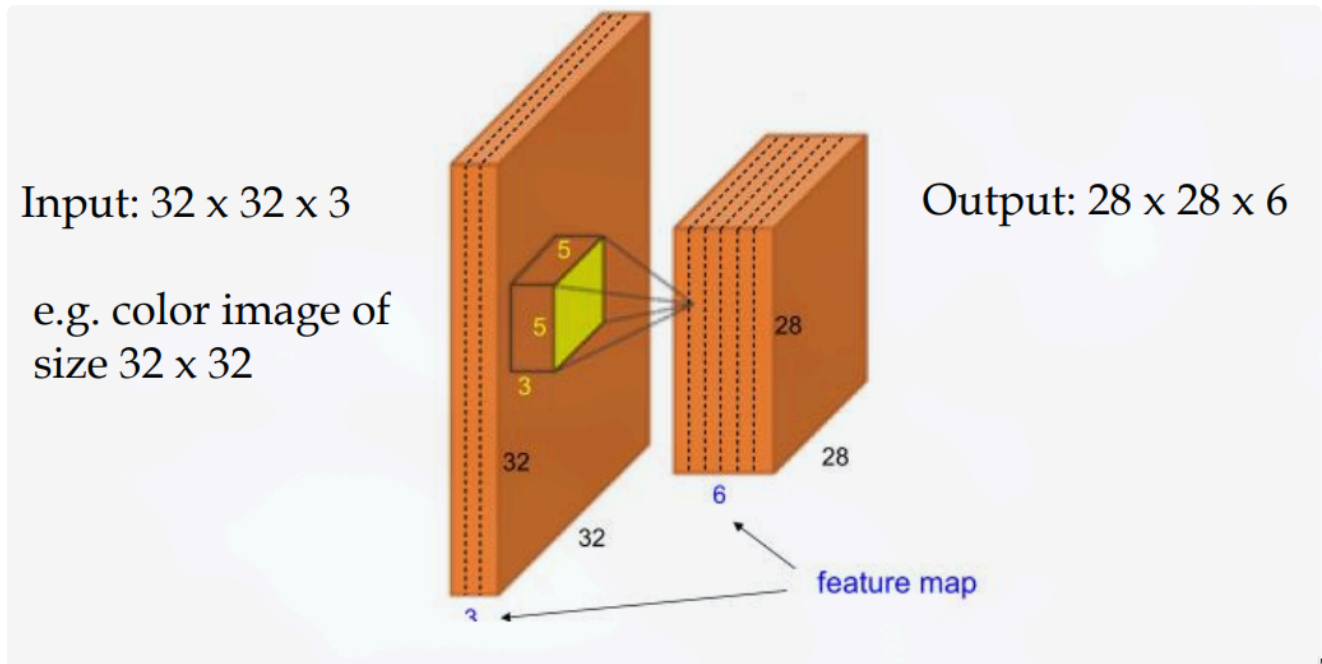
The neurons of each level of a CNN are organised in 3D grids, also called **volumes**:



NOTE

The *depth dimension* identifies **different feature maps**.

This visualisation is of great help to understand how the **kernels (filters)** operate:



FEATURE MAP

A **feature map** is simply a "slice" of neurons at the same depth.

Thus, in the above example we have **3 input feature maps** and **6 output feature maps**.

The weights are **shared at the feature map level**: neurons in the same feature map **process different portions** of the input volume in the same way.

Thus, **each feature map** can be seen as the **result of a specific input filter**.

In the previous example, the *number of connections* between the two levels is $(28 \times 28 \times 6) \times (5 \times 5 \times 3) = 352800$, but the **total number of weights** is just $6 \times (5 \times 5 \times 3 + 1) = 456$ thanks to *weight sharing* (the +1 is the bias).

INSIGHT ON FEATURE MAPS

Notice that a **feature map** is basically a **2D spatial grid** that shows **where a specific learned pattern exists in the input image**.

Crucially, the **pattern represented by a feature map** heavily depends on its **depth** in the network, because its neurons have **different receptive field sizes**. More precisely, as we **move from the input to the output layer**, the **receptive field** of neurons **increases**: intuitively, this is because each neuron in a feature map at a certain depth is the *result of a convolution of neurons in the previous*

feature map, which are themselves results of convolutions of even more neurons in the preceding layers.

This forces the feature maps to **represent increasingly complex and abstract concepts**:

- **Early layers** will **detect edges, corners and colours**.
- **Middle layers** may pick up **simple shapes** (*combinations of edges*).
- **Deep layers** will detect **complete objects** (*combinations of simpler shapes*).

So far we have assumed that a filter is **scrolled on the input volume in unit steps** (that is, the filter is applied by *moving the filter one neuron to the right at each step*)

In reality, a **larger step (stride)** can be used.

This operation **reduces the size of the feature maps in the output volume** and, consequently, it also reduces the number of connections.

Applying **small strides** (e.g. 2, 4) on the **initial levels of the network** allows to achieve **better (computational) efficiency** at the **expense of a slight reduction in accuracy**.

Furthermore, we can also **add a padding of zero values** to the border of the **input volume** to **bring it to a desired dimension**; this way, we can **process images** that are **smaller than the expected input**.

Let w_{out} be the *width of the output feature map*, w_{in} be the *width of the input feature*, w_f be the *width of the filter*, w_p the *padding size* and s be the *stride (step size)*.

Then, we clearly have:

$$w_{\text{out}} = \frac{w_{\text{in}} + 2w_p - w_f}{s} + 1$$

Obviously, the *same applies to the height*.

POOLING LAYERS

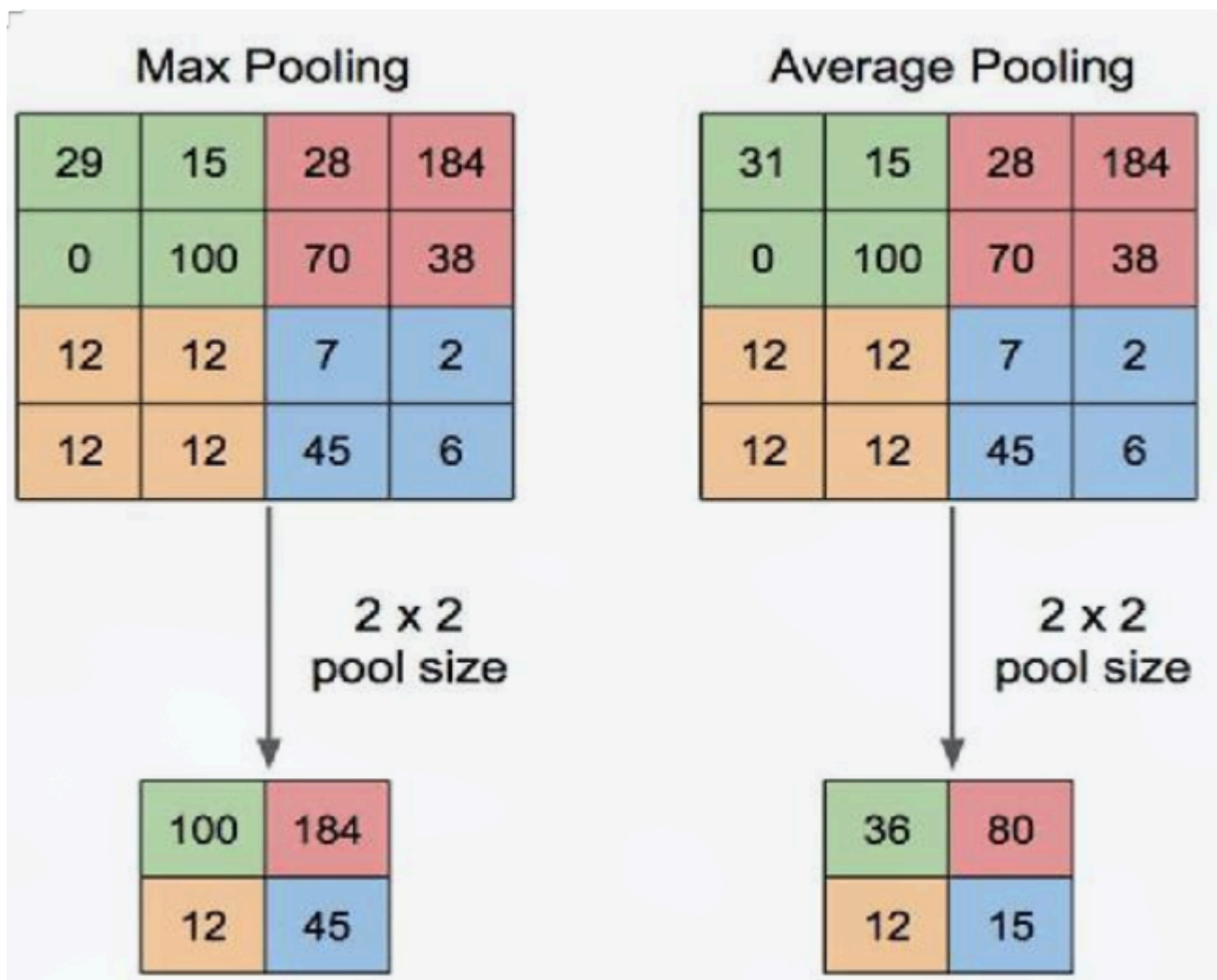
CNNs often include levels known as **pooling layers**:

POOLING LAYER

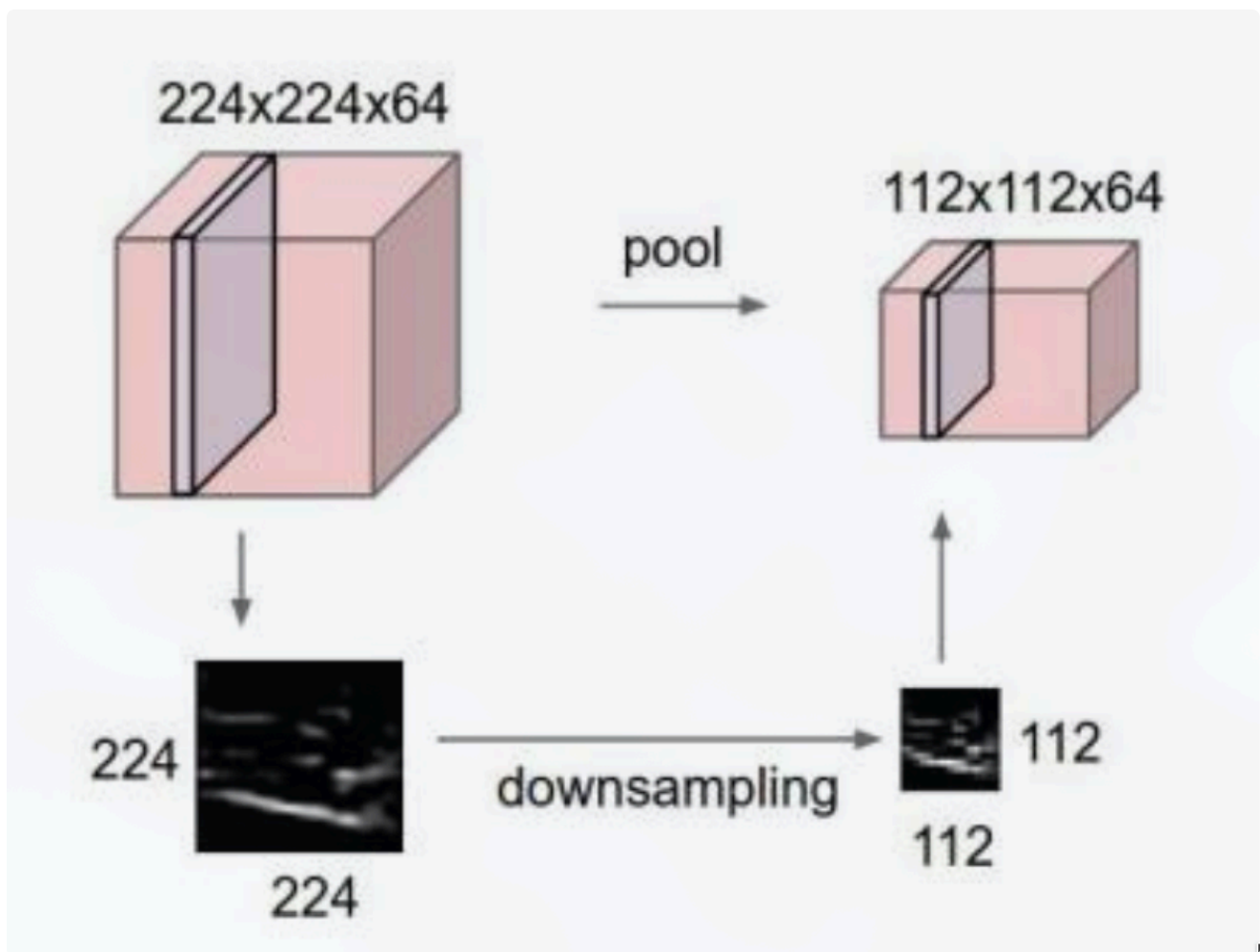
A pooling layer **aggregates information in the input volume**, producing **smaller feature maps**.

Aggregation **operates** (generally) **within each feature map**, so that the feature maps in the output volumes are **simply "shrunk"** versions of the input feature maps, while **remaining the same in number**.

The most commonly used **aggregation operators** are **average** (avg) and **maximum** (max): both *"rather" invariant under small translations*.



In the above example, pooling is applied with a **2x2 filter** and a **stride** $s = 2$. Using the same method on a **224x224x64 volume**, we obtain a **112x112x64 output volume**, as shown below:



CNNs AS TRANSLATION INVARIANT, CONSTRAINED MLPs

In the paragraphs above we introduced the CNN as a **neural network architecture** that is particularly **well suited for image processing** thanks to the fact that it **essentially implements convolutions**.

This explanation can be rather uninteresting to some, who might crave a **more mathematically sound justification** for the network's internal workings.

For instance, one may wonder:

Why specifically choose convolutions and not some other processing technique?

Thus, the aim of this paragraph is to show that **convolutions** naturally **emerge** when we want the network to satisfy some **key requirements**.

"DERIVING" CNNs

Let us go back to the **root problem**: we want to **develop a deep neural network** specifically to **process images**.

From such a network, we **expect two key characteristics**:

1. **TRANSLATION INVARIANCE**: in the *early layers*, our network should *respond similarly to the same "patch", regardless of where it appears in*

the image.

2. **LOCALITY PRINCIPLE:** the *early layers* of the network should *focus on local regions*, with *no regard for the contents of the image in distant regions*.

Notice that these requirements are reasonable if we consider that:

1. When **detecting objects**, we should **not be** overly **concerned** with their **precise locations** in the image: we **simply want to detect them**. Hence the *translation invariance* requirement.
2. Looking at the **entire image at once** introduces "**distractions**" that make it **harder to detect specific objects**, hence the *locality principle*.

To start off, we can consider an MLP with *two-dimensional images* $\mathbf{X}$ as *inputs*, and their **immediate hidden representations** $\mathbf{H}$ similarly represented as *matrices*.

For simplicity reasons, we assume both $\mathbf{X}$ and $\mathbf{H}$ have the **same shape**.

Let $[\mathbf{X}]_{i,j}$ and $[\mathbf{H}]_{i,j}$ denote the **pixel** at location (i, j) in the input image and hidden representation, respectively.

Consequently, to have **each of the hidden units receive input from each of the input pixels** (notice this is simply a *fully-connected MLP*), we would switch from using *weight matrices* (as we did with MLPs) to **representing our weights as fourth-order tensors** W .

Suppose $\mathbf{U}$ contains the **biases** for each pixel in $\mathbf{H}$; we can thus formally express the **fully-connected layer** as:

$$[\mathbf{H}]_{i,j} = [\mathbf{U}]_{i,j} + \sum_k \sum_l [W]_{i,j,k,l} [\mathbf{X}]_{k,l}$$

Notice that the element $[W]_{i,j,k,l}$ represents the weight to pixel (i, j) in $\mathbf{H}$, coming *from* pixel (k, l) in $\mathbf{X}$.

Now, we set $k = i + a$, $l = j + b$ and $[V]_{i,j,a,b} = [W]_{i,j,i+a,j+b}$, obtaining:

$$[\mathbf{H}]_{i,j} = [\mathbf{U}]_{i,j} + \sum_a \sum_b [V]_{i,j,a,b} [\mathbf{X}]_{i+a,j+b}$$

The indices a and b run over **both positive and negative offsets** to cover the entire image. This change of variable is a **simple shift of perspective**: that is, we compute each location (i, j) in $\mathbf{H}$ by **summing over pixels centred** around (i, j) and weighted by $[V]_{i,j,a,b}$.

Now that notation is agreed upon, let us think about **what translation invariance means**.

Notice that "[...] *regardless of where it appears in the image*" means that a **shift in the input $\mathbf{X}$** should simply lead to a **shift in the hidden representation $\mathbf{H}$** .

This is **only possible** if V and U **do not actually depend on the pixel location (i, j)** , i.e., we have:

$$[V]_{i,j,a,b} = [\mathbf{V}]_{a,b} \quad \mathbf{U} = u$$

As a result, we can **simplify** the definition of $\mathbf{H}$:

$$[\mathbf{H}]_{i,j} = u + \sum_a \sum_b [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b}$$

🔗 OBSERVATION

This is exactly a **convolution**!

The only problem is that, even though $[\mathbf{V}]_{a,b}$ needs **many fewer parameters** than $[V]_{i,j,a,b}$, the convolution is calculated **across the entire image and not portion-wise**.

Thus, let us look at how we can fix this detail.

To do so, we simply need to think **what the locality principle implies**.

As motivated above, we believe we **should not look very far** from location (i, j) in order to gain relevant information to assess **what is going on** at $[\mathbf{H}]_{i,j}$.

This means that **outside some range** $|a| > \Delta$ or $|b| > \Delta$ we **should set** $[\mathbf{V}]_{a,b} = 0$.

Equivalently, we can rewrite $[\mathbf{H}]_{i,j}$ as follows:

$$[\mathbf{H}]_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b}$$

🔗 CONCLUSION

Finally, we have obtained the **"window-wise" convolutions** that we were looking for, with $\mathbf{V}$ being the **convolution kernel**.

Thus, we have given a **mathematical argument** for the **need of convolutions**: when we impose *restrictions on how the network should behave*, convolutions *simply emerge from these conditions*.

SOME CNN EXAMPLES

Now that we have a good grasp of how all the **single pieces** of CNNs work, we take a look at some **example architectures** to properly understand this kind of networks.

The main key points to address are **network depth** and **kernel sizes**, but, as we will see, further design choices are possible depending on the task.

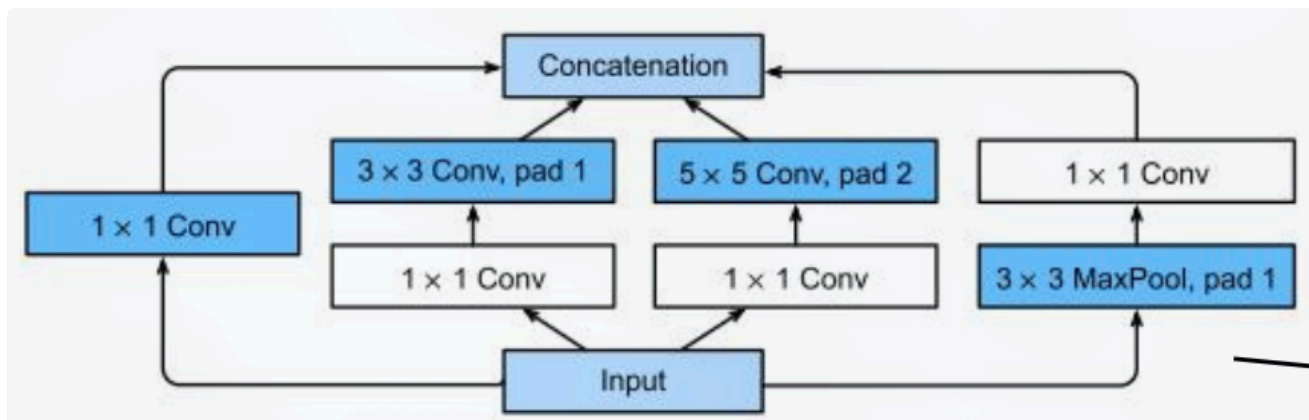
GOOGLNET

In 2014 **GoogleNet** won the *ImageNet* competition, proposing a structure that emphasised the strength of **repeated blocks**.

As introduced above, one focus of the paper was to address the question of **which size is best for convolution kernels**: previous popular networks employed choices ranging from 1×1 to 11×11 .

An insight given by the paper was that sometimes it can be advantageous to employ a **combination of variously-sized kernels**.

This idea is reflected in the **basic convolutional blocks** of GoogleNet: **inception blocks** (likely named due to a quote from the movie *Inception*, which sparked a viral meme).



As we can see in the figure, an inception block consists of **four parallel paths**.

The first three use convolutional layers with window sizes of 1×1 , 3×3 and 5×5 to **extract information from different spatial sizes**.

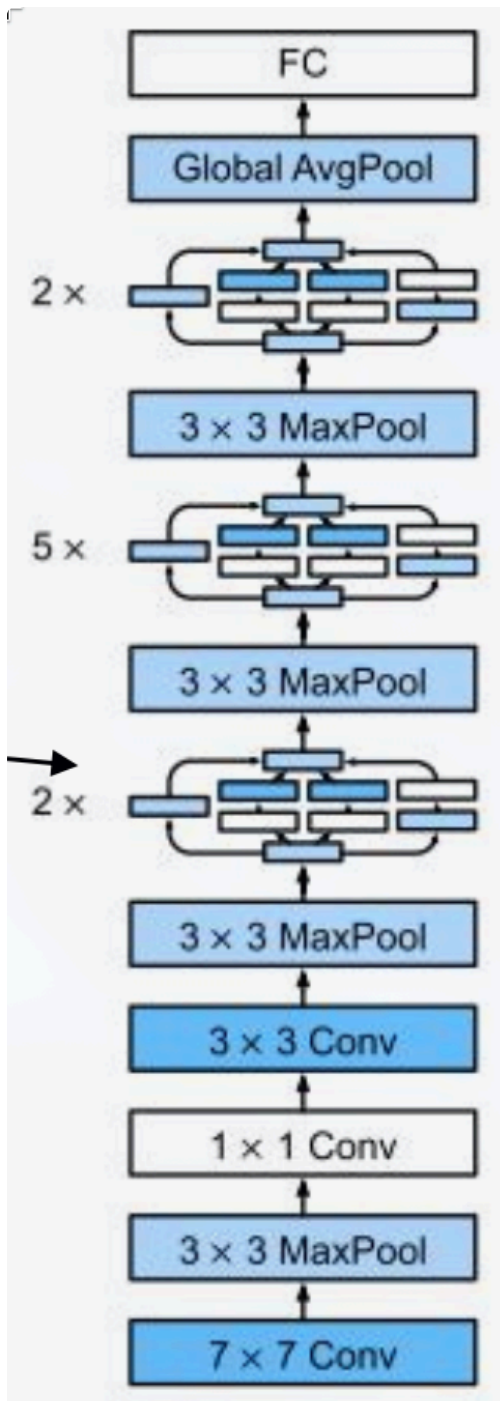
The middle two paths then perform a 1×1 convolution on the input to **reduce the number of channels** (matching the number of output channels of the first path).

The fourth path uses 3×3 maximum pooling layer followed by a 1×1 convolutional layer, which again matches the number of output channels from the other paths.

All four paths use **appropriate padding** so that the output all have the same dimensions.

Finally, the four outputs along each path are **concatenated** along the **channel dimension** and comprise the block's output.

These blocks are **repeated** throughout the network, resulting in the following architecture:



FINAL LAYER

The last layer takes a *feature map* of size 7×7 with 1024 channels and **averages it** to size 1×1 .

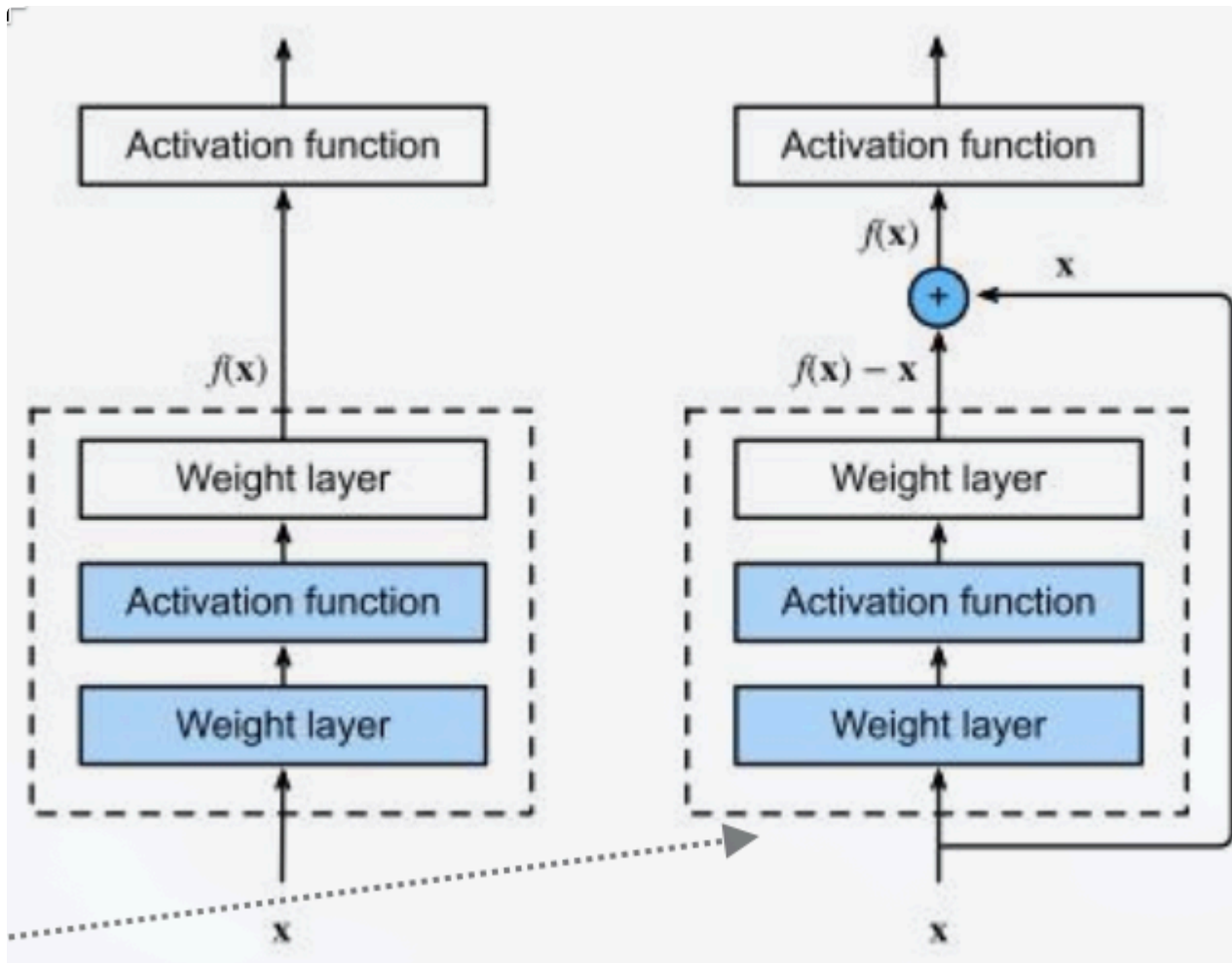
Thus, the **output** is a **vector of size 1024**.

To gain some intuition on why this network works so well, we need to consider the **combination of the filters**: they explore the image in a *variety of filter sizes*, which means that **details at different extents** can be **recognised efficiently** by filters of different sizes.

RESNET

He et al. were working on very deep computer vision models when they proposed their **ResNet** architecture. At the heart of their proposal is the idea that **every additional layer** in a deep network should be **easy to train into the identity function** $f(x) = x$ if we want to **expand the function class** reachable by the network (See > [1.2 - DEEP NEURAL NETWORKS > FUNCTION CLASSES](#)).

This consideration led to a surprisingly simple solution: a **residual block**. The illustration below compares this block (right) to a "classic" convolution block (left).



We denote the block input by x and assume that the **desired underlying mapping** we want to obtain by learning is $f(x)$, to be used as **input to the activation function** on the top.

We can clearly see that a **classic** convolution block (left) **must directly learn the full mapping** $f(x)$, while the **residual block** (right) **needs only learn the residual mapping** $f(x) - x$ (and hence the name), because the original input is carried over and added to the output of the new block.

💡 IMPLICATION ON THE NETWORK'S FUNCTION CLASS

This guarantees that **adding an extra layer** of this kind simply **expands the original function class**; that is, $\mathcal{F}_{\text{old}} \subset \mathcal{F}_{\text{new}}$.

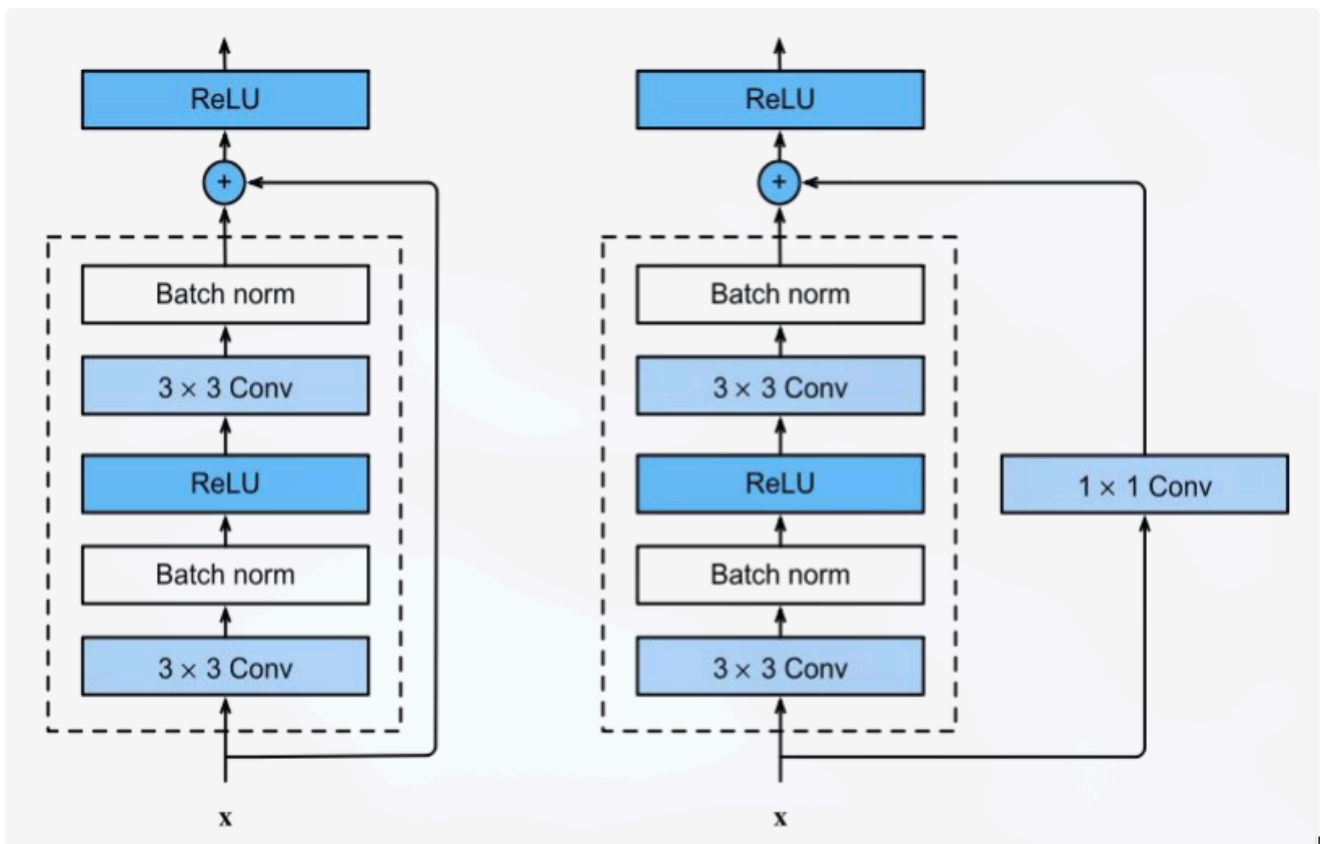
In fact, we can **obtain the same result** as the original network by pushing the new weights to zero.

If these new weights learn non-zero values instead, the network can **learn a wider range of functions**.

ResNet follows a full 3×3 convolutional layer design.

The residual block has **two 3×3 convolutional layers** with the **same number of output channels** as the input x , each followed by a **batch normalisation layer** and a **ReLU activation function** (see the figure below (left)).

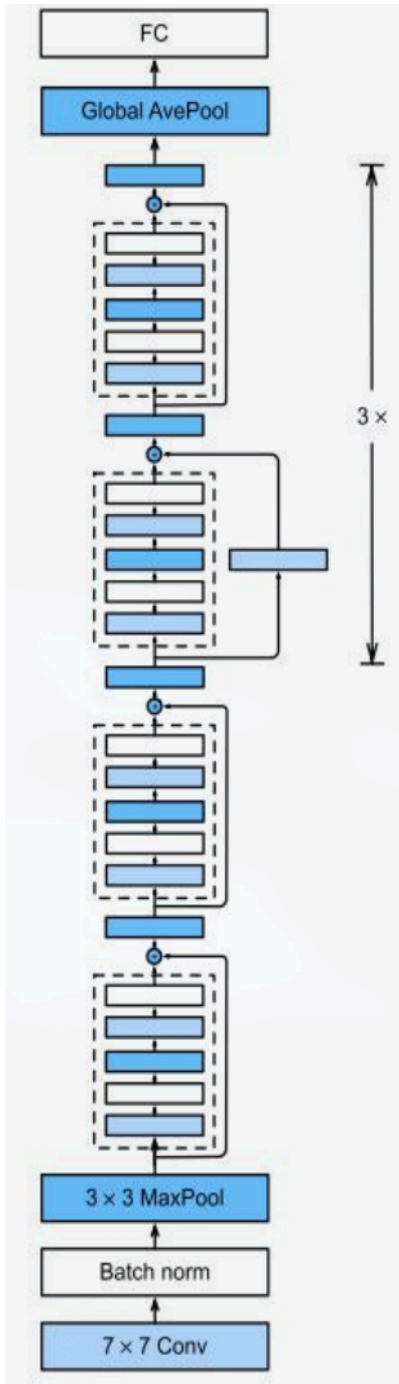
If we want to **change the number of output channels**, we need to introduce an **additional 1×1 convolutional layer** to transform the input into the **desired shape** (see the figure below (right)).



By configuring the number of residual blocks and their arrangements we can obtain **different ResNet models**, usually denoted by the *number of layers* in their architecture:

NUMBER OF LAYERS	NUMBER OF PARAMETERS
ResNet-18	11.174M
ResNet-34	21.282M
ResNet-50	23.521M
ResNet-101	42.513M
ResNet-152	58.157M

The figure below illustrates the ResNet-18 architecture.



RESNET SCALING

Residual blocks allow the **gradients to flow more directly** through the network during training.

As a result, **deep ResNets** are effectively **easier to train** compared to traditional architectures with the same depth.

DENSENET

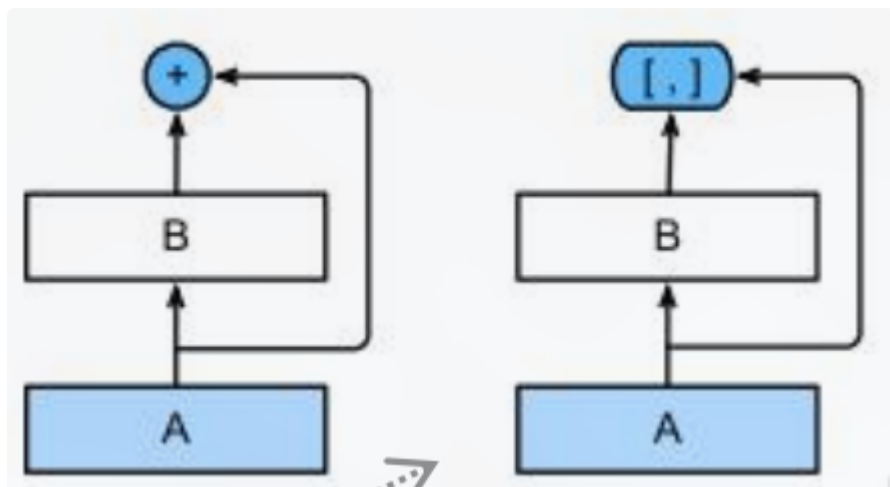
ResNet significantly changed the view of how to parametrise functions in deep networks.

DenseNet (*Dense convolutional network*), developed by *Huang et al.* is, to some extent, the **logical extension** of this.

ResNet **decomposes** f into a simple **linear term** and a more complex **nonlinear term**, but what if we **want to capture information beyond just two terms**?

DenseNet is precisely a **possible solution** to this problem.

The main difference between ResNet (left in the figure below) and DenseNet (right in the figure below) is that in the latter case **outputs are concatenated** (denoted by $[,]$) rather than added:



As a result, we perform a mapping from $\mathbf{x}$ to its values after **applying an increasingly complex sequence of functions**:

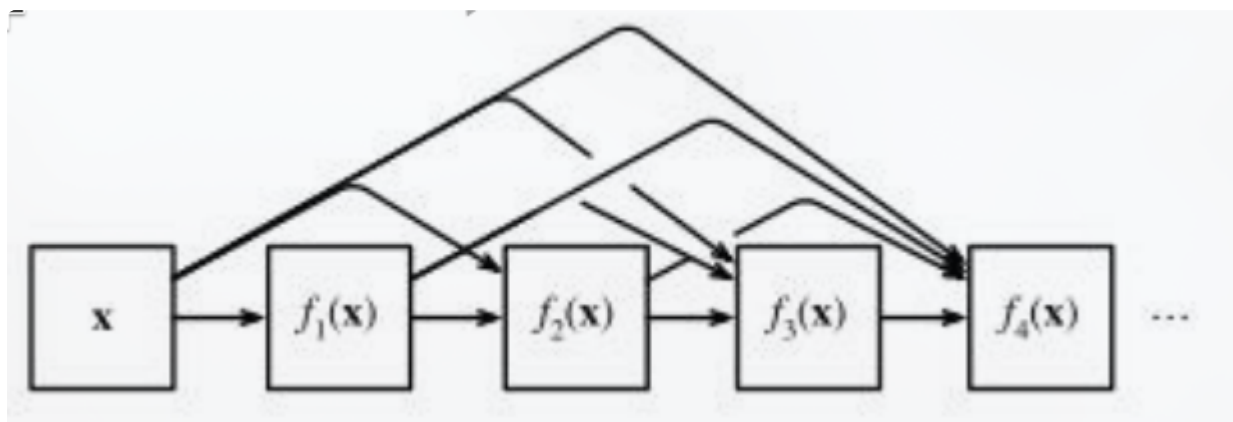
$$\mathbf{x} \rightarrow [\mathbf{x}, f_1(\mathbf{x}), f_2([\mathbf{x}, f_1(\mathbf{x})]), f_3([\mathbf{x}, f_1(\mathbf{x}), f_2([\mathbf{x}, f_1(\mathbf{x})])), \dots]$$

In the end, all of these functions are **combined in a MLP** to **reduce the number of features** again.

In terms of implementation, this is quite simple: rather than adding terms, we simply concatenate them.

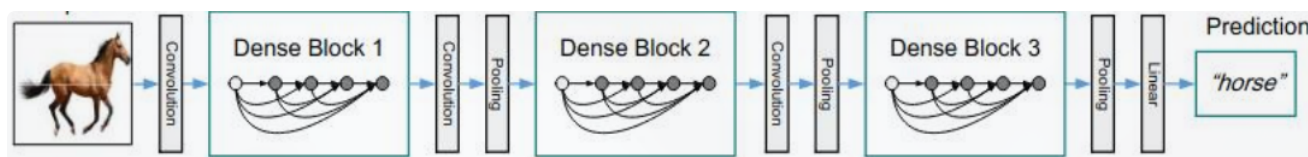
The name DenseNet arises from the fact that the **dependency graph between variables** becomes quite **dense**.

The last layer of such a chain is **densely connected to all previous layers**, as shown below:



The **main components** of a DenseNet are **dense blocks** and **transition layers**. The former define *how the inputs and outputs are concatenated*, while the latter control

the number of channels so that it is not too large.



TRANSITION LAYERS

In the figure above, *transition layers* are the ones in between the *dense blocks*: they **downsample feature maps** by using **batch normalisation**, **ReLU**, 1×1 **convolution layers** (to reduce the number of channels) and **pooling layers**.

ENZYNET

In the [INPUT CHANNELS](#) paragraph we briefly stated that "*non image data can be converted to a tensor to be fed to a CNN*"; it is thus worth taking a look at an example of a CNN that **works with non-image data**.

EnzyNet is a convolutional neural network **classifier** that *predicts the Enzyme Commission number of enzymes based on their voxel-based spatial structure* (to be more precise, the network only predicts the *top-level* EC number).

CONTEXT: ENZYME COMMISSION NUMBER

EC number is a **numerical classification scheme** for enzymes, based on the *chemical reactions they catalyse*: if different enzymes catalyse the same reaction, then they receive the same EC number.

Before looking at the network's architecture, which is rather simple, let us dive into **how enzymes are represented as tensors**.

It has been established that **3D structure is fundamental to determine the function of enzymes**: accordingly, enzymes are represented as a **binary volumetric shape** with volume elements (*voxels*) fitted in a cube V of a fixed grid size l with respect to the three dimensions.

In particular, each **voxel** $(x, y, z) \in [0, l - 1]^3$ takes **value 1** if the backbone (only carbon, nitrogen and calcium atoms are considered to obtain the representation) of the **enzyme passes through the voxel**, 0 otherwise.

At the end, the results look like the ones shown below:

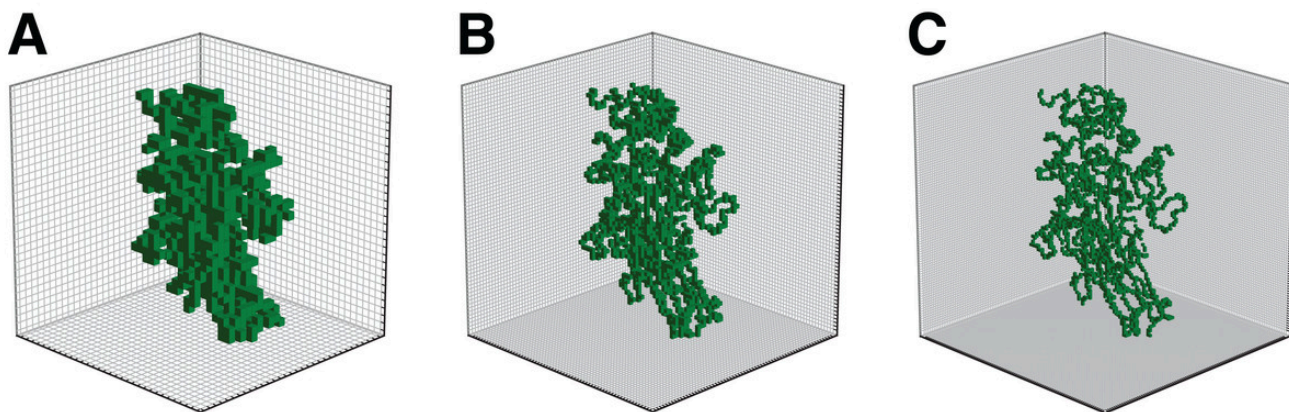


FIGURE ABOVE

The figure illustrates the **volumetric shape** of enzyme 2Q3Z for different grid sizes (A: $l = 32$. B: $l = 64$. C: $l = 96$).

PREPROCESSING

Before feeding the representations to the network, some **preprocessing steps** are required (preprocessing is often accompanied by data augmentation, the importance of which has already been introduced in > [1.1 - MORE ON NETWORK TRAINING > DATA AUGMENTATION](#)).

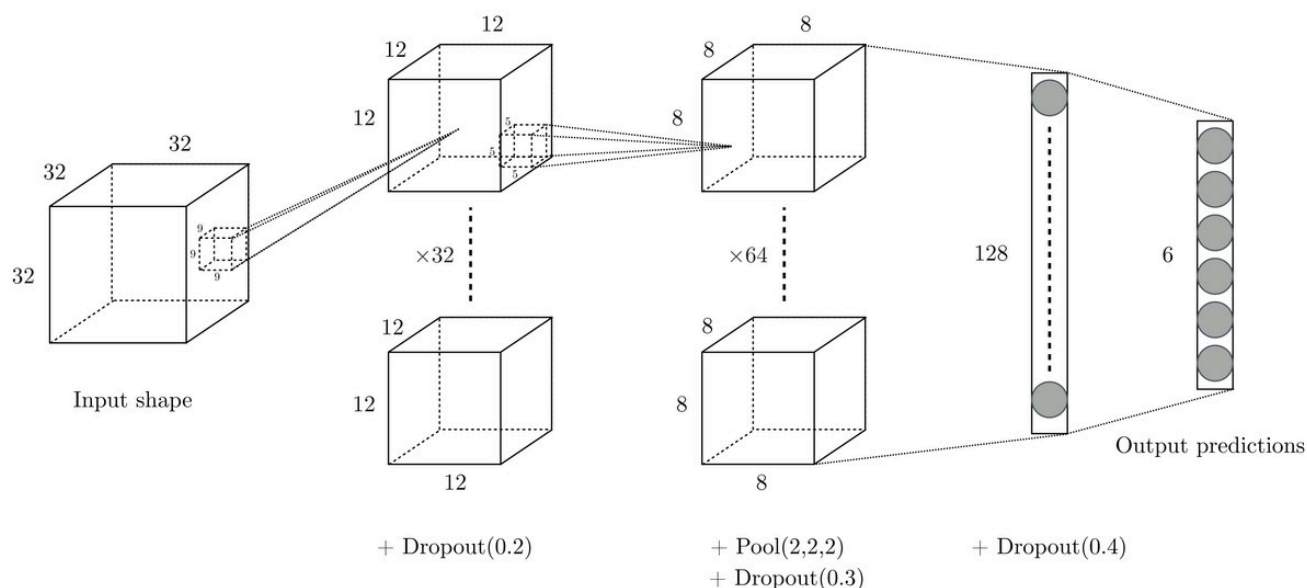
In particular, because enzymes do *not possess any absolute spatial orientation*, all of the enzymes in the data set are represented in a coordinate system such that:

- The **centre** lies on the enzyme's **barycentre**.
- The **axes** are calculated via **principal component analysis**.

Furthermore, each enzyme may be **randomly flipped** with respect to one or more axes to **account for differences in spatial orientation**.

Finally, **outliers** are **removed** from the volume.

Though the interesting part (how non-image data can be represented as a tensor) is over, it is still worth looking at the **EnzyNet architecture**, shown in the figure below:



It is a simple **two-layer architecture**: input volumes of size $32 \times 32 \times 32$, containing enzymes encoded in voxels, first go through a **convolutional layer** of 32 filters of size $9 \times 9 \times 9$ with stride 2.

Then, a **second convolutional layer** of 64 filters of size $5 \times 5 \times 5$ with stride 1 is used, followed by a **max-pooling layer** of size $2 \times 2 \times 2$ with stride 2.

Finally, there are **two fully-connected layers** of 128 and six (the number of classes) hidden units respectively, concluded by a **softmax layer** that **outputs class probabilities** corresponding to the six first-level EC numbers.

OTHER CONCEPTS

The previous paragraphs laid the foundational knowledge about convolutional neural networks and presented some concrete example networks to better understand how those concepts come together in practice.

Obviously, there are many more details that need to be considered when designing a CNN, lots of which can be incredibly subtle and would require entire paragraphs to be discussed exhaustively.

Nonetheless, this last paragraph aims at giving a brief introduction to some of these.

TRANSFER LEARNING

Training **complex CNNs** on **large datasets** is **computationally expensive**. In addition, training the network on a **new problem** requires **another large labeled training set**, which may often not be available.

On the other hand, once **training is over**, it is **much faster** to perform **forward propagation** to classify the patterns.

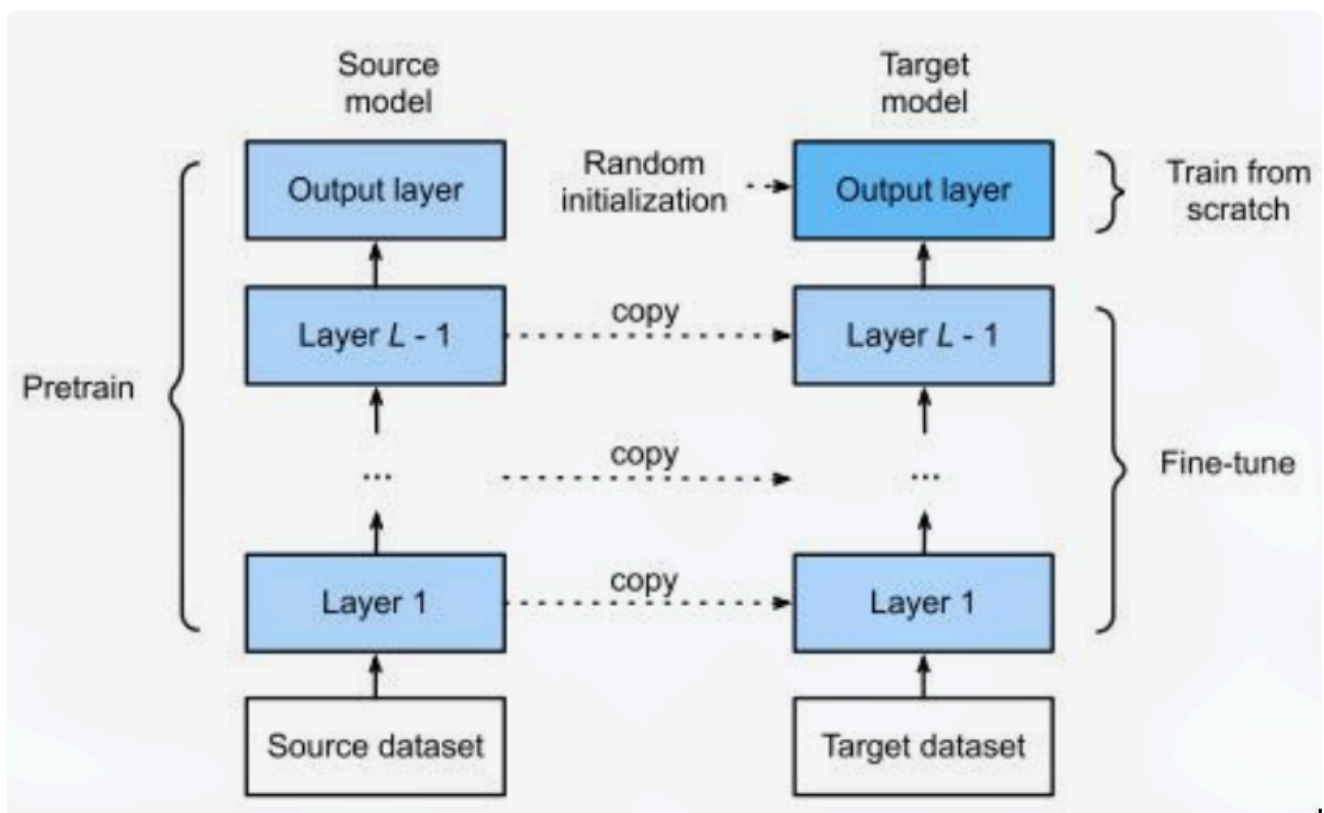
Thus, when we want to **use an existing network** on a **new problem** we can use **transfer learning techniques** such as **fine-tuning** and **deep features**.

Starting with an existing *pre-trained* network, we can adapt it to solve **similar problems** by:

1. **Replacing the output layer** with a new one (the new number of classes might be different).
2. **Copying the weights** for all the intermediate layers, and **randomly initialising** the weights in the **last layer**.
3. Performing **new training iterations** to **optimise the weights** on the **new task**.

The practicality of this approach stems from the fact that the new training iterations required are **far less** than what is needed to **train** the network **from scratch**.

Another incredibly useful consequence is that the **new dataset** does **not need to be very large**.



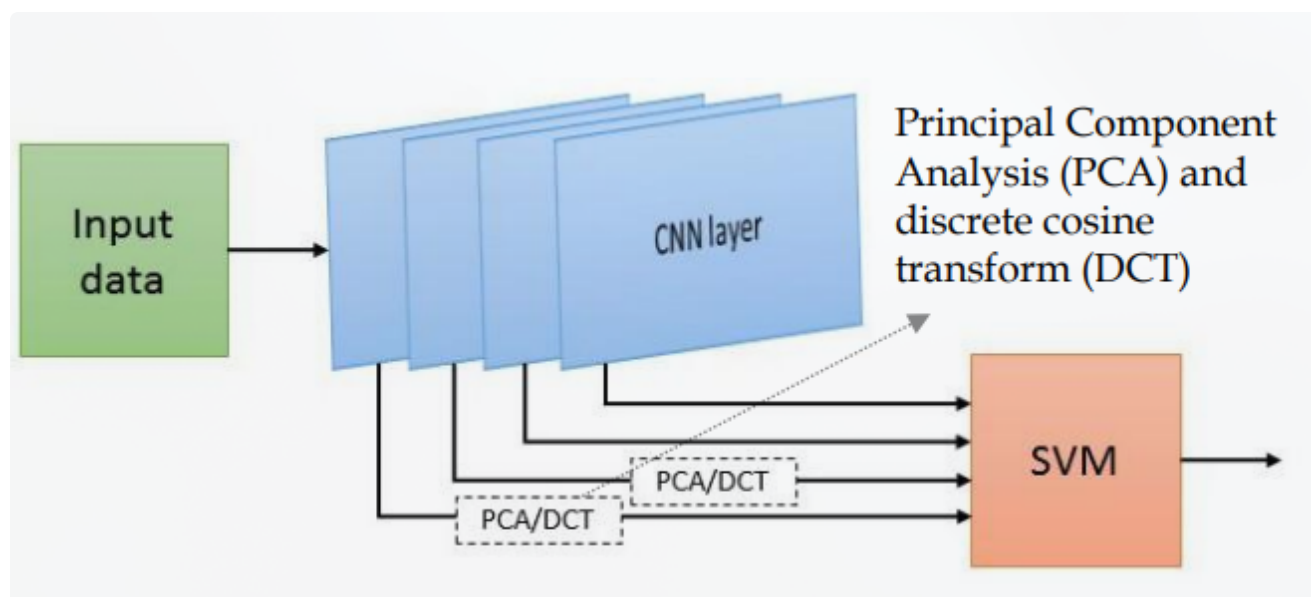
📋 DEEP FEATURES

An existing network can be used to **extract features** generated by its **intermediate layers**.

These features can then be used to **train an external classifier** (e.g. Support Vector Machine) to **classify the patterns of the new application domain**.

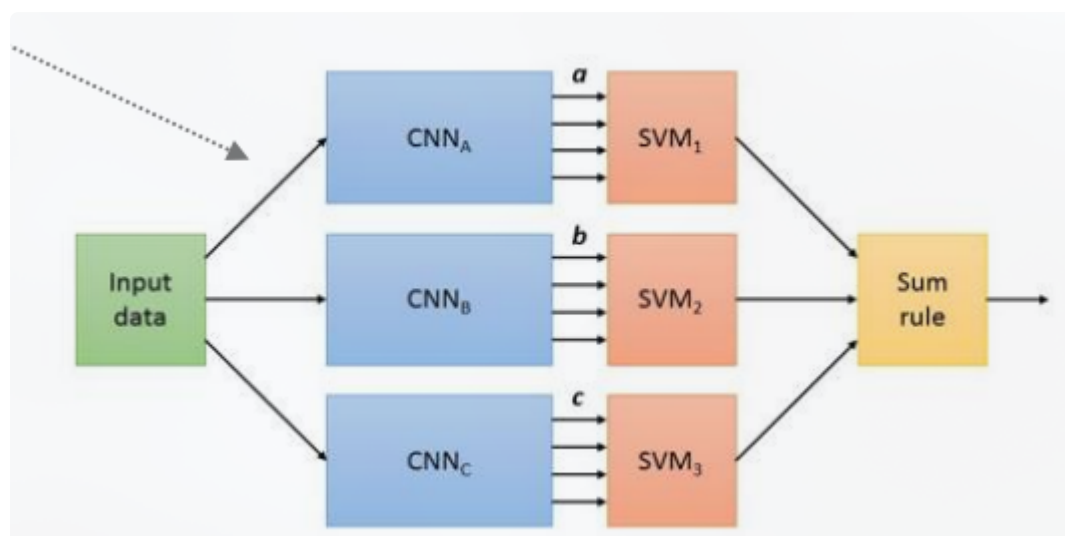
It is important to notice that it is **possible** to use the **features** generated by **early layers** in the network, but this **may be problematic** because of the **very high dimensionality**

of these features. Thus, it is common practice to employ **dimensionality reduction** techniques when extracting features from the early layers of a network.



Furthermore, instead of concatenating the various feature vectors **extracted from different layers** and/or from **different networks** (e.g. different architectures), we can **train a different classifier** (e.g. SVM) for **each single feature extraction** and then **combine the results** (e.g. by summing them).

It's been shown that, in many applications, different topologies carry **partially independent information**, therefore **combining different CNNs** allows for **increased performance**.



AUTO ENCODER

When working with deep neural networks it is often interesting to look at the **representations (encoding)** for a **set of data** learned by the network, because these can play a huge role in the correctness and accuracy of the output.

Consequently, engineers have worked on specific solutions to the **encoding problem**, such as **auto encoders**.

The aim of an auto encoder (**AE**) is to **learn a representation** for a set of data, typically for *dimensionality reduction*, by **training the network to ignore signal "noise"**.

📖 AUTO ENCODERS

AEs transform the **input** into a *new representation* (called **latent-space representation**) and then **reconstruct the output** from this representation.

In particular, an AE is composed by:

- An **encoding function** $E(\mathbf{x}) : \mathbb{R}^n \rightarrow \mathbb{R}^k$ ($k < n$) outputting a latent representation s .
- A **decoding function** $D(s) : \mathbb{R}^k \rightarrow \mathbb{R}^n$ computing the reconstructed output $\mathbf{o}$.

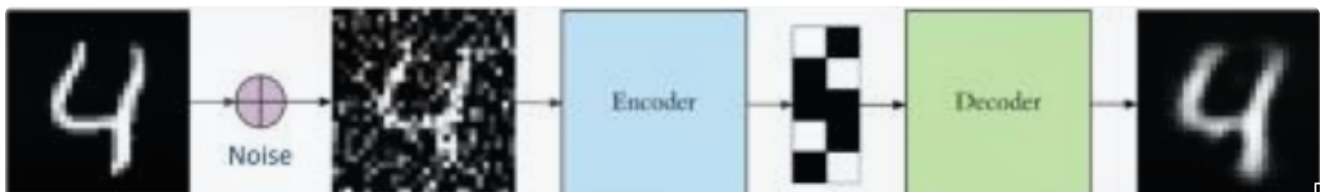
From the definition above, it is simple to deduce that **simplest form** of an AE is a **neural network with 3 layers**: an **input layer**, an **intermediate layer** and an **output layer** with the **same number of neurons as the input**.

Clearly, when building an autoencoder, our **goal** is to **minimise the reconstruction error** $\mathcal{L}(\mathbf{x}, \mathbf{o})$; that is, minimise the *difference between the input and the output*. Notice that this introduces a problem:

⚠️ POTENTIAL RISK

The AE may learn the **identity function**, so the **output simply equals the input**.

This can be prevented by "**corrupting**" the input data on purpose by **adding noise** or **masking some of the values**: as a result, the AE is **forced to recover the original noise-free data**.



Obviously, this "*input corruption*" is **only** performed during **training**.

Alternative methods of preventing AEs from learning the identity function also exist; one of them being **sparse autoencoders**.

📖 SPARSE AUTO ENCODERS

Spare AEs introduce a **sparsity constraint** by **adding a term** to the **loss function**, such that **only a fraction of the neurons** become **active**:

$$\mathcal{L}(\mathbf{x}, \mathbf{o}) + \Omega(\mathbf{h})$$

Where Ω is a **penalty function** on **hidden layer activations** $\mathbf{h}$.

The penalty term has a **sparsifying effect** (similar to that of regularisation techniques) which forces the AE to **represent each input** as a **combination of a small number of neurons**.

At this point, one may ask "*but why do we want to prevent the AE from learning the identity function?*".

The answer lies in the **reason** that leads us to **want to use an AE**: the **hope** is that the **latent representation** will take on **useful properties** (some interesting hidden structure) during **training** that will **then improve generalisation** on test and validation.

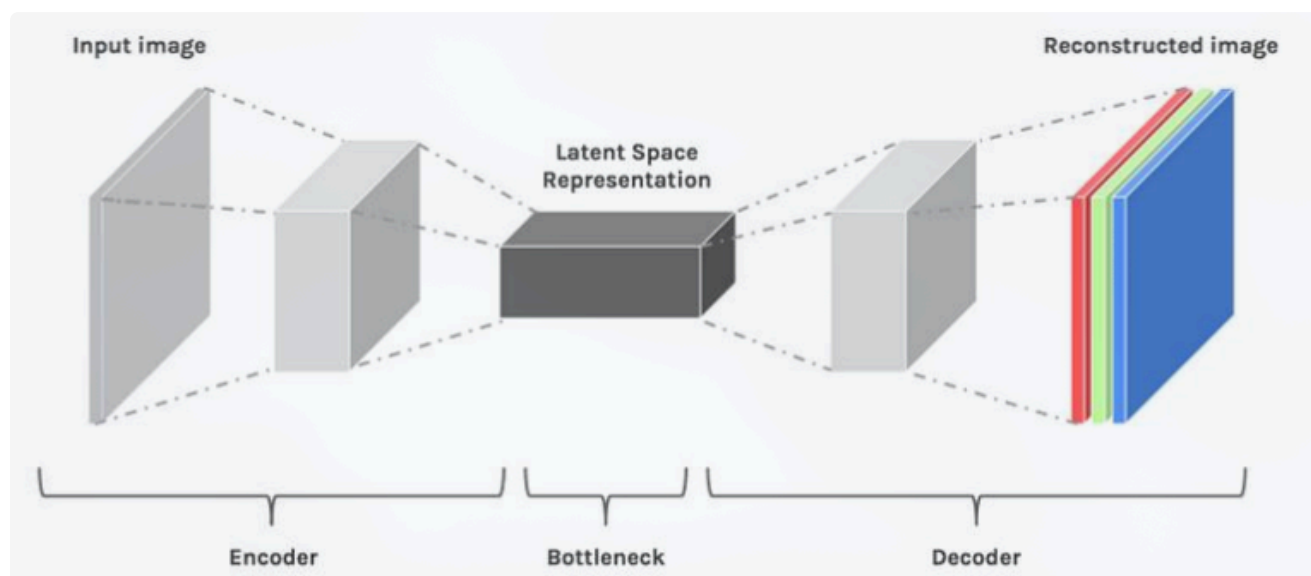
This can be achieved, for example, if **similar patterns** are **positioned closer** to one another in the latent space.

With this in mind, it is easy to see **why dimensionality reduction** is **important** in AEs:

💡 BOTTLENECK

If the original patterns are represented in a **high-dimensional space**, even **similar patterns** may be quite **distant from each other**, but by **reducing the dimensionality** we can "**compact**" them together.

This is why the term "**bottleneck**" is often used.



Intuitively, the bottleneck "**forces**" the network to learn an **efficient representation** for the input that **better encodes** its semantics, so that it is *easier to extract useful information from the training set*.

✍ NOTE

Sometimes, engineers are only interested in obtaining an *efficient encoder*. In such cases, the whole AE is trained, and at the end the *decoder* is simply discarded.

ADVERSIAL EXAMPLES

The concept of **adversial examples** was first proposed by *Szegedy et al.* in a study about "unconventional" phenomena of neural networks.

Among these, one is the **vulnerability** of neural networks in **image classification**: an **attacker** generates **examples** with **offensive characteristics** by adding carefully crafted, small-magnitude, and imperceptible perturbations to the clean samples (thus obtaining the **adversial examples**).

Such examples can make even neural networks with high classification confidence and accuracy **output the wrong results**.

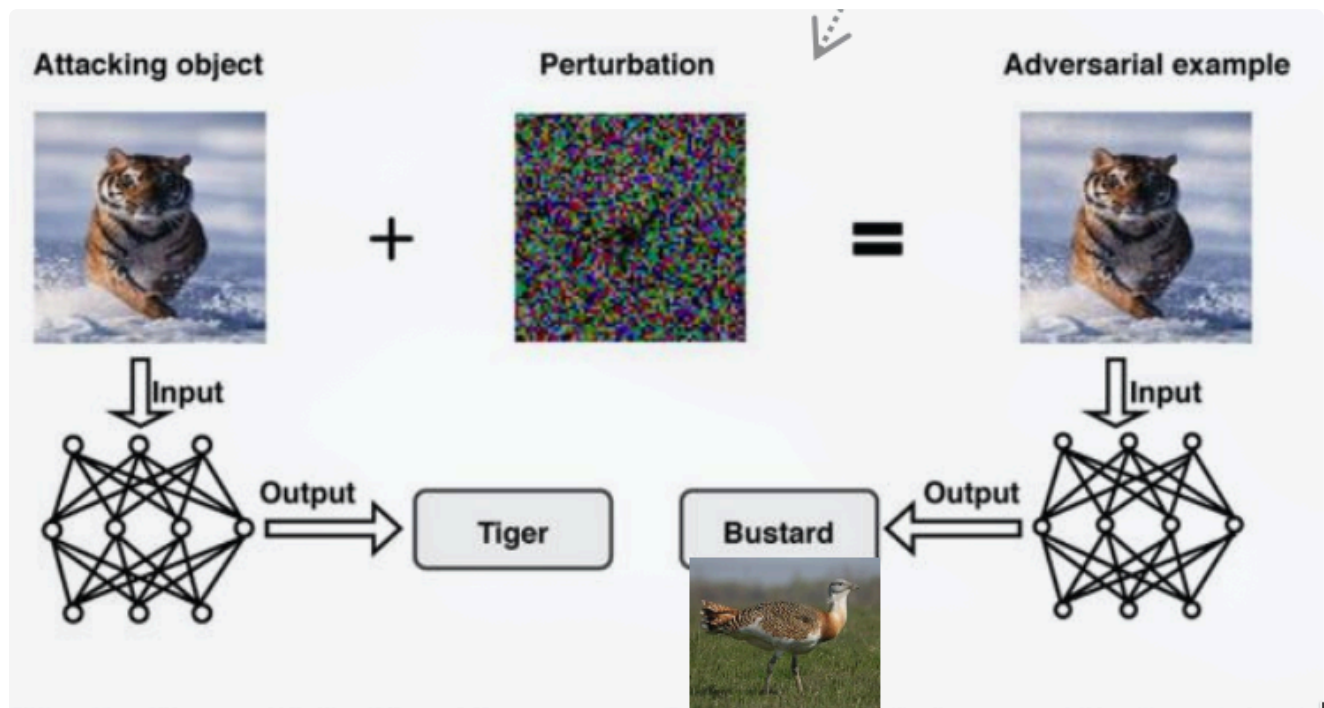
📖 MATHEMATICAL STATEMENT OF ADVERSIAL EXAMPLES

Mathematically, **crafting an adversial example** consists in finding the **minimum perturbation** δ for an image that successfully **throws off the model predictions**:

$$\min \|\delta\| \text{ s.t. } f(x + \delta) \neq l, x + \delta \in [0, 1]^m$$

Where x is the *clean sample*, δ is the perturbation, $f(\cdot)$ is the *model's prediction* and $l = f(x)$.

The figure below illustrates the general pipeline for generating adversial examples:



Furthermore, adversial examples attacks can be **non-target** or **targeted**:

📋 NON-TARGET VS TARGETED ATTACKS

- **Non-target** attack: the aim is to make the model output **any category other than the correct one**. Thus, the attack is successful whenever $f(x + \delta) \neq l$.
- **Targeted** attack: the aim is to make the model **output the category specified by the attacker** (y). Thus, the attack is successful if $f(x + \delta) = y \neq l$.

Targeted attacks are **harder to achieve** and may **require information about the model**, which may be obtained via a **query attack**.

📖 QUERY ATTACK

A query attack is a type of adversarial attack where an attacker attempts to **exploit vulnerabilities** in a machine learning model by **making queries to the model** and **using the responses to craft adversarial examples**.

This way, adversarial examples can be **crafted** in such a way that works for the **specific model** being targeted.